CONFIDENTIAL — MANUFACTURER PARTNER

VAPI Manufacturer Integration Specification

Hardware Integration Requirements for VAPI-Compatible Game Controllers

Version: 1.0 — Phase 228

Date: April 2026

Author: VAPI Protocol Engineering

Reference Device: Sony DualShock Edge (CFI-ZCP1)

Distribution: Authorized manufacturer partners under NDA only

---

Table of Contents

14. Appendix B: Reference Contracts

---

1. Executive Summary

The **Verified Autonomous Physical Intelligence (VAPI)** protocol is the world's first cryptographic anti-cheat system for competitive gaming that operates at the **physics layer**. Rather than relying on kernel-level software introspection or server-side heuristics, VAPI captures and cryptographically attests the raw physical signals produced by a human player's interaction with a game controller—including inertial measurement, touchpad contact geometry, trigger resistance profiles, and micro-tremor signatures—to produce tamper-evident **Proof of Authentic Controller (PoAC)** records.

For manufacturers, VAPI integration provides three strategic benefits:

15. **Hardware Certification:** Controllers that pass the Physical Hardware Certification Initiative (PHCI) earn on-chain attestation, distinguishing them from uncertified competitors.
16. **Attested Tier Premium Positioning:** Only controllers meeting full PITL L0–L6 requirements qualify for the Attested tier, commanding premium market positioning in competitive gaming.
17. **Data Marketplace Revenue:** Anonymized, privacy-preserving biometric models derived from VAPI-instrumented controllers generate revenue through the ioSwarm data marketplace, with manufacturer revenue share.

This document defines the **complete hardware, firmware, cryptographic, and manufacturing requirements** for integrating VAPI capabilities into game controllers. It is targeted at hardware engineers, firmware teams, manufacturing operations, and certification engineers. The reference device throughout this specification is the **Sony DualShock Edge (CFI-ZCP1)**.

IMPORTANT

All values marked as **FROZEN** in this specification are protocol-locked and must be reproduced exactly. Deviation from frozen values will result in PoAC record rejection by the verifier network.

2. BOM Delta vs. Reference Device (Sony DualShock Edge CFI-ZCP1)

The following table identifies the Bill of Materials (BOM) delta—components that must be **added to** or **verified present in** a standard game controller to achieve VAPI compatibility. The reference baseline is the Sony DualShock Edge (CFI-ZCP1).

Component	Part Number / Spec	Purpose	Required for Attested?	Required for Standard?	Est. Unit Cost Delta
Secure Element	ATECC608A (Microchip)	Stores ECDSA-P256 private key, device identity; tamper-resistant cryptographic operations	**YES**	Recommended	$0.50 – $1.00
IMU (6-axis)	3-axis gyro: 16-bit, ±2000 dps, 1000 Hz 3-axis accel: 16-bit, ±16 g, 1000 Hz	Physiological tremor detection (8–12 Hz), micro-movement classification, PITL L2/L3 layers	**YES**	**YES**	Already present in DualShock Edge
Touchpad	Capacitive,	Primary	**YES**	Optional	Already

Component	Part Number / Spec	Purpose	Required for Attested?	Required for Standard?	Est. Unit Cost Delta
	1920×1080, multi-touch (2-point)	biometric discriminator (`touchpad_spatial_entropy`); PITL L4 layer			present in DualShock Edge
Adaptive Triggers	L2/R2 adaptive resistance, 3 modes	PITL L6 active challenge-response; measures physical resistance to trigger depression	**YES**	Not required	Already present in DualShock Edge
GSR Grip Module (optional addon)	ESP32-S3 BLE, 2 electrodes/grip, 100 Hz, 16-bit, 1 kΩ–10 MΩ	Galvanic skin response measurement for enhanced biometric fidelity	Optional	Optional	$3.00 – $5.00 (external addon)
Firmware Flash Storage	Additional 256 KB	VAPI firmware module storage	**YES**	**YES**	$0.05 – $0.10
USB-C (1000 Hz polling)	USB-C, HID 1000 Hz, 64-byte report	High-frequency transport for Attested tier; timing precision for PoAC chain integrity	**YES**	250 Hz BLE acceptable	Already present in DualShock Edge

NOTE

For controllers already matching DualShock Edge capabilities (touchpad, adaptive triggers, 6-axis IMU, USB 1000 Hz), the **primary BOM addition is the**

ATECC608A secure element (~$0.50–$1.00 unit cost) plus 256 KB additional firmware flash storage (~$0.05–$0.10). Total incremental BOM cost: approximately **$0.55–$1.10 per unit**.

3. Hardware Component Requirements

3.1 Secure Element (ATECC608A or Equivalent)

Parameter	Requirement
Cryptographic Algorithm	ECDSA-P256 key generation and signing
Hash Support	SHA-256
Key Storage	Minimum 16 slots
Slot 0	Device identity private key (generated on-chip, **never exported**)
Slot 1	VAPI protocol signing key
Slot 2	Session ephemeral key derivation
Slots 3–5	Manufacturer certificate chain storage
Interface	I2C, 1 MHz clock
Tamper Detection	Active shield, voltage monitoring, temperature monitoring
Operating Temperature	−40 °C to +85 °C
Side-Channel Resistance	Constant-time signing operations required

CRITICAL REQUIREMENT

The device identity private key in Slot 0 must be **generated on-chip** and must **never be exported** from the secure element under any circumstances. Any implementation that allows private key extraction will fail PHCI certification.

3.2 Inertial Measurement Unit (6-axis)

Parameter	Requirement
Gyroscope	3-axis, 16-bit resolution, ±2000 dps full-scale range
Gyroscope Sample Rate	1000 Hz minimum
Accelerometer	3-axis, 16-bit resolution, ±16 g full-scale range
Accelerometer Sample Rate	1000 Hz minimum
Noise Floor	Sufficiently low for physiological tremor detection in the 8–12 Hz band
Reference	DualShock Edge uses integrated 6-axis IMU meeting all above specifications

The IMU is the foundation of PITL layers L2 (gyroscope micro-tremor analysis) and L3 (accelerometer grip pattern classification). The 8–12 Hz physiological tremor band is the primary frequency range for human identity verification; excessive sensor noise in this band will degrade classification accuracy and may prevent PHCI certification.

3.3 Touchpad (Attested Tier — Mandatory)

Parameter	Requirement
Technology	Capacitive
Resolution	Minimum 1920 × 1080
Multi-Touch	2-point minimum

Parameter	Requirement
Report Rate	Synchronized with USB polling (1000 Hz)
PITL Layer	L4 — `touchpad_spatial_entropy`

IMPORTANT

`touchpad_spatial_entropy` is the **#1 inter-player biometric discriminator**, as confirmed in Phase 143 analysis. This feature alone provides the highest separation ratio between distinct human players. Controllers without a touchpad **cannot qualify for Attested tier**.

3.4 Adaptive Triggers (Attested Tier — Mandatory — L6 Layer)

Parameter	Requirement
Functionality	Variable resistance **measurement** (not just haptic feedback)
Trigger Coverage	Both L2 and R2 must support adaptive resistance modes
Effect Modes	Minimum 3 trigger effect modes
PITL Layer	L6 — Active challenge-response

L6 Purpose: The L6 layer uses adaptive triggers to perform active challenge-response measurements. The controller applies variable physical resistance to trigger depression and measures the player's force response profile. **Software injection cannot simulate physical resistance to trigger depression**—this is the definitive hardware-layer anti-cheat mechanism.

WARNING — NON-QUALIFYING TRIGGER TYPES

The following trigger technologies do **NOT** qualify for Attested tier (L6):

- **Impulse triggers** (Xbox) — vibration only, no variable resistance measurement
- **Hair triggers** (Scuf) — reduced travel, no resistance modulation
- **Mecha-tactile triggers** (Razer) — tactile click, no adaptive resistance
- **Standard triggers** — no active resistance capability

Controllers with these trigger types are limited to Standard tier (L0–L5 maximum).

3.5 Transport Interface

Transport	Polling Rate	Tier Eligibility	Additional Requirements
USB	1000 Hz	Attested + Standard	HID report size: 64 bytes
BLE	250 Hz	Standard only	GATT HID service
Proprietary Wireless	1000 Hz equivalent	Attested + Standard	Latency < 2 ms, jitter < 0.5 ms at 1000 Hz

4. Secure Element Provisioning Flow

4.1 Factory Provisioning (Manufacturing Line)

The following steps must be executed for every controller unit on the manufacturing line. Steps must be performed in order. Steps 1–6 are irreversible after Step 6 (configuration zone lock).

Step	Action	Details
1	Generate ECDSA-P256 keypair on ATECC608A	On-chip generation. Private key never leaves secure element. Stored in Slot 0.
2	Extract public key from Slot 0	64 bytes (uncompressed without 0x04 prefix) or 65 bytes (with 0x04 prefix).
3	Compute `device_id`	`device_id = keccak256(public_key)` — **NOTE: Uses keccak256, NOT SHA-256.**
4	Sign device certificate with manufacturer's intermediate CA	X.509 certificate binding `device_id` to public key, signed by manufacturer CA.
5	Write manufacturer certificate chain to Slots 3–5	Slot 3: device cert. Slot 4: intermediate CA cert. Slot 5: root CA cert hash.
6	Lock configuration zone (**irreversible**)	Prevents modification of key slots and configuration. Cannot be undone.
7	Record in provisioning database	Store: `device_id`, `public_key`, certificate hash, manufacturing timestamp, unit serial number.
8	Flash VAPI firmware module to controller	256 KB firmware module written to dedicated flash partition.

4.2 First-Use Activation (End User)

Step	Action	Details
1	Controller connects via USB to player's PC	PC must be running the VAPI bridge application.

Step	Action	Details
2	Bridge reads `device_id` and public key from secure element	Via HID feature report or dedicated VAPI HID usage page.
3	Bridge verifies manufacturer certificate chain	Validates device cert → intermediate CA → VAPI root CA trust anchor.
4	Bridge registers device on IoTeX L1	Calls `VAPIioIDRegistry` contract. DID: `did:io:0x{device_id}`
5	Bridge registers hardware certification	Calls `VAPIHardwareCertRegistry` contract. `certLevel`: 1 (basic), 2 (standard), or 3 (attested).
6	Device enters “enrolled” state	Ready for calibration sessions. Minimum N≥50 sessions required for Attested tier threshold derivation.

4.3 Sample Provisioning Script (C Pseudocode)

```
/*  * VAPI Factory Provisioning — ATECC608A via I2C  * WARNING: This is pseudocode for illustration only.  * Adapt to your I2C HAL and ATECC608A library.  */  #include <atecc608a.h> #include <keccak256.h> #include <x509.h> #define SLOT_DEVICE_KEY      0 #define SLOT_PROTOCOL_KEY    1 #define SLOT_SESSION_KEY     2 #define SLOT_DEVICE_CERT     3 #define SLOT_INTER_CA_CERT   4 #define SLOT_ROOT_CA_HASH    5  int vapi_factory_provision(i2c_handle_t i2c, const x509_cert_t *mfr_ca) {     uint8_t pubkey[64];     uint8_t device_id[32];     x509_cert_t device_cert;     int rc;      /* Step 1: Generate ECDSA-P256 keypair in Slot 0 (on-chip) */     rc = atecc_genkey(i2c, SLOT_DEVICE_KEY, pubkey);     if (rc != ATECC_OK) return rc;      /* Step 2: Public key now in pubkey[64] (no 0x04 prefix) */      /* Step 3: Compute device_id = keccak256(pubkey)      * NOTE: keccak256, NOT SHA-256 */     keccak256(pubkey, 64, device_id);      /* Step 4: Sign device certificate with manufacturer intermediate CA */     rc = x509_create_device_cert(&device_cert, pubkey, device_id, mfr_ca);     if (rc != X509_OK) return rc;      /* Step 5: Write certificate chain to Slots 3-5 */     rc = atecc_write_slot(i2c, SLOT_DEVICE_CERT, device_cert.der, device_cert.der_len);     if (rc != ATECC_OK) return rc;     rc = atecc_write_slot(i2c, SLOT_INTER_CA_CERT, mfr_ca->der, mfr_ca->der_len);     if (rc != ATECC_OK) return rc;      rc = atecc_write_slot(i2c, SLOT_ROOT_CA_HASH,                           mfr_ca->root_hash, 32);     if (rc != ATECC_OK) return rc;      /* Step 6: Lock configuration zone (IRREVERSIBLE) */     rc = atecc_lock_config(i2c);     if (rc != ATECC_OK) return rc;      /* Step 7: Record in provisioning database
```

```
*/     provisioning_db_record(device_id, pubkey, &device_cert);      /* Step 8: Flash VAPI firmware module (handled by separate flash tool) */     return VAPI_PROVISION_OK; }
```

5. Firmware API and PoAC Builder Reference

5.1 PoAC Wire Format (228 Bytes — FROZEN)

> **PROTOCOL-FROZEN VALUE**
>
> The PoAC wire format is **exactly 228 bytes**. This format is frozen and must **never be modified**. All multi-byte integers are **big-endian**. All floating-point values are **IEEE 754 double-precision (64-bit)**.

Offset (bytes)	Size (bytes)	Field	Type	Description
0	8	`timestamp_ns`	uint64	Nanosecond-precision monotonic timestamp
8	1	`inference_code`	uint8	Classification result (see code table below)
9	1	`confidence_byte`	uint8	0–255 maps linearly to 0.0–1.0
10	4	`session_id`	uint32	Unique session identifier
14	4	`sequence_number`	uint32	Monotonically increasing per-session counter
18	2	`feature_dim`	uint16	Number of

Offset (bytes)	Size (bytes)	Field	Type	Description
				feature dimensions (currently **13**)
20	2	`reserved`	—	Must be `0x0000`
22	8	`anomaly_score`	float64	Anomaly detection score (IEEE 754 double)
30	8	`continuity_score`	float64	Biometric continuity score (IEEE 754 double)
38	96	`features[12]`	float64[12]	12 feature values × 8 bytes each (IEEE 754 double)
134	16	`device_id`	bytes	`keccak256(pubkey)` truncated to 128 bits
150	14	`reserved_padding`	—	Must be `0x00 × 14`
164	64	`signature`	bytes	ECDSA-P256 signature: `r || s` (32 bytes each)

Total: 228 bytes. Body = bytes[0:164]. Signature = bytes[164:228].

CRITICAL — CHAIN LINK FORMULA

`record_hash = SHA-256(raw[0:164])` — body ONLY, **never the full 228 bytes**. This hash is the chain link that connects consecutive PoAC records into a tamper-evident chain. Signing or hashing the full 228-byte record (including the signature) will produce incorrect chain links and fail verification.

Inference Code Table (FROZEN):

Code	Constant	Meaning
`0x00`	NOMINAL	Normal human input detected
`0x28`	DRIVER_INJECT	Driver-level input injection detected
`0x29`	WALLHACK	Wallhack behavioral pattern detected
`0x2A`	AIMBOT	Aimbot behavioral pattern detected
`0x2B`	TEMPORAL_BOT	Temporal bot pattern (inhuman timing regularity)
`0x30`	BIOMETRIC_ANOMALY	Biometric profile does not match enrolled identity
`0x31`	IMU_PRESS_DECOUPLED	IMU and button press events are statistically decoupled
`0x32`	STICK_IMU_DECOUPLED	Stick movement and IMU movement are statistically decoupled

5.2 PoAC Builder API (C-Style Interface)

```
/*  * PoAC Builder API — Firmware Reference Interface  * All implementations MUST conform to this interface.  */  typedef struct {     uint64_t timestamp_ns;       /* Offset 0:   monotonic nanosecond timestamp     */ uint8_t  inference_code;     /* Offset 8:   classification result */     uint8_t  confidence;         /* Offset 9:   0-255 maps to 0.0-1.0 */     uint32_t session_id;         /* Offset 10:  unique session identifier */     uint32_t sequence_number;    /* Offset 14:  monotonic per-session counter       */     uint16_t feature_dim;        /* Offset 18:  feature count (currently 13)        */     /* 2 bytes reserved at offset 20 (0x0000) */     double   anomaly_score;      /* Offset 22:  anomaly detection score */     double   continuity_score;   /* Offset 30:  biometric continuity score */     double   features[12];       /* Offset 38:  feature vector (96 bytes) */     uint8_t  device_id[16];      /* Offset 134: keccak256(pubkey)[0:16] */     /* 14 bytes reserved_padding at offset 150 (0x00 x 14) */ } PoACBody; /* 164 bytes when serialized to wire format */  /**  * Build a complete 228-byte PoAC record from a populated body struct.  * Serializes body to big-endian wire format, then signs.  *  * @param body   Populated PoACBody struct * @param out    Output buffer, minimum 228 bytes  * @return       0 on success, negative error code on failure  */ int vapi_poac_build(PoACBody *body, uint8_t out[228]);  /**  * Sign body bytes[0:164] using ATECC608A
```

```
secure element.  * Produces ECDSA-P256 signature as r || s (64 bytes).  *  *
@param body   Serialized body bytes (164 bytes, big-endian)  * @param sig
Output signature buffer (64 bytes)  * @return       0 on success, negative
error code on failure  */ int vapi_poac_sign(const uint8_t body[164], uint8_t
sig[64]);  /**  * Compute chain link hash: SHA-256(body[0:164]).  * This hash
links consecutive PoAC records.  *  * @param body   Serialized body bytes
(164 bytes)  * @param hash   Output hash buffer (32 bytes)  * @return       0
on success, negative error code on failure  */ int vapi_record_hash(const
uint8_t body[164], uint8_t hash[32]);  /**  * Verify chain integrity by
comparing prev_hash with  * the computed record_hash of the previous record's
body.  *  * @param prev_hash   Expected previous record hash (32 bytes)  *
@param body        Current record's body bytes (164 bytes)  * @return
0 if chain is valid, -1 if broken  */ int vapi_chain_verify(const uint8_t
prev_hash[32], const uint8_t body[164]);
```

5.3 Sensor Sampling Requirements

Sensor / Interface	Sample Rate	Notes
HID Report Polling (USB)	1000 Hz	Required for Attested tier
HID Report Polling (BLE)	250 Hz	Acceptable for Standard tier only
Gyroscope	1000 Hz (native rate)	Synchronized with HID polling where possible
Accelerometer	1000 Hz (native rate)	Synchronized with HID polling where possible
Touchpad	Report on every USB poll when active	Include X, Y coordinates and touch state
Adaptive Triggers	Report resistance value on every poll	Include current resistance mode and measured force
Timestamp	Nanosecond precision	System clock, monotonic (must not go backward)

6. Transport Timing and Jitter Requirements

6.1 Attested Tier Transport Requirements

Parameter	Requirement	Measurement Method
Polling Rate	1000 Hz (USB) or equivalent proprietary wireless	10,000-sample measurement window
p99 Latency	< 2 ms	End-to-end HID report timing
Jitter (std dev)	< 0.5 ms	Standard deviation of inter-report intervals
Packet Loss	0%	Over 10,000 consecutive reports
Clock Drift	< 1 ppm	Measured against reference clock

6.2 Standard Tier Transport Requirements

Parameter	Requirement
Polling Rate	250 Hz (BLE) or 1000 Hz (USB)
p99 Latency	< 5 ms
Jitter (std dev)	< 1 ms
Packet Loss	< 0.1%

6.3 BLE-Specific Notes

- BLE 250 Hz thresholds are **structurally different** from USB 1000 Hz thresholds. Calibration models are transport-specific and not interchangeable.
- Separate calibration track required — the composite key includes transport type as a discriminating field.
- N ≥ 50 BLE sessions required per controller model for threshold derivation.
- **BLE transport is currently disabled in the protocol** (`bt_transport_enabled = false`). Manufacturers should implement BLE support for future activation but should not rely on BLE for current Attested tier certification.

7. Manufacturing Test Harness and Certification Checklist

7.1 Hardware Verification Tests

The following tests must be executed for every controller unit on the manufacturing line. All tests must pass before the unit is shipped.

Test ID	Test Name	Description	Pass Criteria
HW-001	ATECC608A Communication	I2C read/write to secure element, serial number verification	Successful read of serial number; I2C ACK on all transactions
HW-002	Key Generation	Generate test keypair, verify ECDSA-P256 signature	Signature verifies against extracted public key
HW-003	IMU Self-Test	Gyro/accel self-test registers, noise floor measurement	Self-test pass; noise floor within IMU datasheet limits
HW-004	Touchpad Calibration	Multi-touch accuracy, spatial resolution verification	Touch coordinates accurate to ±2 px at 1920×1080; both touch points registered
HW-005	Adaptive Trigger Test	Resistance measurement across full travel, mode switching	Resistance values span full range; all 3 modes switchable
HW-006	USB Polling Rate	Verify 1000 Hz ±1% over 10,000 samples	Mean interval: 1.0 ms ±0.01 ms; no dropped reports
HW-007	HID Report Format	Verify report size (64 bytes) and layout correctness	Report size = 64 bytes; all field offsets match profile YAML

7.2 Firmware Verification Tests

Test ID	Test Name	Description	Pass Criteria
FW-001	PoAC Builder	Build 228-byte record, verify byte layout and field offsets	All offsets match Section 5.1 wire format; total size = 228
FW-002	SHA-256 Hash	Compute `record_hash` of known body, compare to test vector	Hash matches test vector from Section 11.1
FW-003	ECDSA-P256 Sign/Verify	Sign known message, verify with public key	Signature verifies; matches expected r, s values (deterministic test)
FW-004	Chain Integrity	Build chain of 10 records, verify `prev_hash` linkage	Each record's prev_hash = SHA-256(previous body[0:164])
FW-005	Timestamp Monotonicity	Generate 1000 records, verify timestamps strictly increasing	Every `timestamp_ns[i+1] > timestamp_ns[i]`
FW-006	Sequence Counter	Verify monotonic increment, no gaps	Every `sequence_number[i+1] = sequence_number[i] + 1`

7.3 PHCI Certification Checklist

PHCI = Physical Hardware Certification Initiative. This is the formal certification process for VAPI-compatible controllers.

Tier 1 — Attested

- PITL layers L0–L6: **Full** implementation required
- Adaptive triggers: **Required** (variable resistance measurement)
- Transport: **1000 Hz USB** or equivalent proprietary wireless

- Calibration: N ≥ 50 sessions for threshold derivation
- Attestation: On-chain attestation via `VAPIHardwareCertRegistry`

Tier 2 — Standard

- PITL layers L0–L5: **Minimum** implementation required
- Transport: 250 Hz BLE acceptable
- Calibration: N ≥ 30 sessions for threshold derivation
- Attestation: Self-registered

PHCI Submission Requirements

18. Submit **3 controller units** for independent testing by VAPI Protocol Engineering.
19. Provide complete **HID report descriptor documentation** (USB and/or BLE).
20. Provide **controller profile YAML** conforming to the schema defined in Section 8.
21. Provide **manufacturer certificate chain documentation** including root CA, intermediate CA, and device certificate formats.
22. Provide **BOM and component sourcing documentation** identifying secure element, IMU, touchpad, and trigger components.

8. Device Capability Manifest Schema

Manufacturers must complete a controller profile in YAML format conforming to the schema below. This profile is required for PHCI certification submission and is used by the VAPI agent fleet for feature availability detection and calibration configuration.

8.1 Schema Definition

```
# VAPI Controller Profile Schema v1.0 # All fields are REQUIRED unless marked
[optional]  identity:   id: string                    # Unique profile
identifier (e.g., "sony_dualsense_edge")   display_name: string          #
Human-readable name   manufacturer: string          # Manufacturer name
model: string                # Model identifier  usb:   vid: hex_uint16
# USB Vendor ID (e.g., 0x054C)   pid: hex_uint16               # USB Product
ID (e.g., 0x0DF2)   interface: uint8              # HID interface number
report_size: uint16          # HID report size in bytes (must be 64)
report_layout: string        # Layout version identifier  ble:
# [optional] — omit if BLE not supported   service_uuid: string          #
GATT HID service UUID   report_char_uuid: string      # HID report
characteristic UUID   name_prefix: string           # BLE advertising name
prefix  transport:   usb_hz: uint16               # USB polling rate (1000
for Attested)   ble_hz: uint16               # BLE polling rate (0 if not
supported)   proprietary: list             # [optional] Proprietary wireless
protocols  pitl_layers:   L0_stick_dynamics: enum       # "full" | "partial"
| "none"   L1_button_timing: enum        # "full" | "partial" | "none"
L2_gyroscope: enum           # "full" | "partial" | "none"
L3_accelerometer: enum       # "full" | "partial" | "none"   L4_touchpad:
enum             # "full" | "partial" | "none"   L5_composite: enum
# "full" | "partial" | "none"   L6_adaptive_trigger: enum     # "full" |
"none"  features:   touchpad:     present: bool     resolution_x: uint16
# e.g., 1920     resolution_y: uint16        # e.g., 1080     multi_touch:
bool     pressure: bool              # Pressure sensitivity supported?
triggers:     type: enum                  # "adaptive" | "impulse" | "hair" |
"mecha" | "standard"     adaptive_l2: bool           # L2 supports adaptive
resistance     adaptive_r2: bool           # R2 supports adaptive resistance
modes: uint8                # Number of trigger effect modes   gyroscope:
present: bool     axes: uint8                 # 3     resolution_bits: uint8
# 16     range_dps: uint16           # 2000     sample_rate_hz: uint16      #
1000   accelerometer:     present: bool     axes: uint8                 # 3
resolution_bits: uint8      # 16     range_g: uint8              # 16
sample_rate_hz: uint16      # 1000   gsr_addon:     supported: bool
# Can accept GSR grip module?  tier_eligibility:   standard: bool
# true if L0-L5 available   attested: bool                # true only if
L6=full AND phci=true AND usb_hz>=1000  calibration:   battery_types: list
# e.g., ["usb_wired", "battery_full", "battery_low"]   feature_dimensions:
uint8     # Number of features (currently 13)   feature_mask: hex_uint16
# Bitmask of enabled features (0x1FFF = all 13)  hid_report_offsets:   lx:
uint8                     # Left stick X byte offset   ly: uint8
# Left stick Y byte offset   rx: uint8                    # Right stick X
byte offset   ry: uint8                   # Right stick Y byte offset   l2:
uint8                     # L2 trigger byte offset   r2: uint8
# R2 trigger byte offset   gyro_x: uint8                 # Gyroscope X byte
offset   gyro_y: uint8                 # Gyroscope Y byte offset   gyro_z:
uint8                 # Gyroscope Z byte offset   accel_x: uint8
# Accelerometer X byte offset   accel_y: uint8               # Accelerometer
Y byte offset   accel_z: uint8               # Accelerometer Z byte offset
touch_x: uint8               # Touchpad X byte offset   touch_y: uint8
# Touchpad Y byte offset   touch_active: uint8           # Touchpad active
flag byte offset   buttons: uint8               # Button state byte offset
phci:   certified: bool              # Has passed PHCI certification?
cert_date: date              # Certification date (ISO 8601)   cert_version:
string          # PHCI version (e.g., "PHCI-2026.1")
```

8.2 Complete Example — Sony DualShock Edge (CFI-ZCP1)

```
identity:   id: "sony_dualsense_edge"   display_name: "Sony DualSense Edge"
manufacturer: "Sony Interactive Entertainment"   model: "CFI-ZCP1"  usb:
vid: 0x054C   pid: 0x0DF2   interface: 3   report_size: 64   report_layout:
"ds_edge_v1"  ble:   service_uuid: "00001812-0000-1000-8000-00805f9b34fb"
report_char_uuid: "00002a4d-0000-1000-8000-00805f9b34fb"   name_prefix:
"DualSense Edge"  transport:   usb_hz: 1000   ble_hz: 250   proprietary: []
pitl_layers:   L0_stick_dynamics: "full"   L1_button_timing: "full"
L2_gyroscope: "full"   L3_accelerometer: "full"   L4_touchpad: "full"
L5_composite: "full"   L6_adaptive_trigger: "full"  features:   touchpad:
present: true     resolution_x: 1920     resolution_y: 1080     multi_touch:
true     pressure: false   triggers:     type: "adaptive"     adaptive_l2:
true     adaptive_r2: true     modes: 3   gyroscope:     present: true
axes: 3     resolution_bits: 16     range_dps: 2000     sample_rate_hz: 1000
accelerometer:     present: true     axes: 3     resolution_bits: 16
range_g: 16     sample_rate_hz: 1000   gsr_addon:     supported: true
tier_eligibility:   standard: true   attested: true  calibration:
battery_types: ["usb_wired", "battery_full", "battery_low"]
feature_dimensions: 13   feature_mask: 0x1FFF    # All 13 features enabled
hid_report_offsets:   lx: 1   ly: 2   rx: 3   ry: 4   l2: 5   r2: 6   gyro_x:
16   gyro_y: 18   gyro_z: 20   accel_x: 22   accel_y: 24   accel_z: 26
touch_x: 33   touch_y: 35   touch_active: 32   buttons: 8  phci:   certified:
true   cert_date: 2026-01-15   cert_version: "PHCI-2026.1"
```

8.3 Example — Xbox Series X Controller (Standard Tier Only)

```
identity:   id: "microsoft_xbox_series_x"   display_name: "Xbox Wireless
Controller (Series X)"   manufacturer: "Microsoft"   model: "1914"  usb:
vid: 0x045E   pid: 0x0B12   interface: 0   report_size: 64   report_layout:
"xbox_sx_v1"  transport:   usb_hz: 1000   ble_hz: 0           # Xbox uses
proprietary wireless, not BLE HID   proprietary:     - name: "Xbox Wireless"
hz: 1000       latency_ms: 2.0  pitl_layers:   L0_stick_dynamics: "full"
L1_button_timing: "full"   L2_gyroscope: "partial"    # Gyro available on
some SKUs only   L3_accelerometer: "partial"   L4_touchpad: "none"        #
No touchpad   L5_composite: "partial"    # Limited by missing L4
L6_adaptive_trigger: "none" # Impulse triggers do NOT qualify  features:
touchpad:     present: false     resolution_x: 0     resolution_y: 0
multi_touch: false     pressure: false   triggers:     type: "impulse"
# Impulse triggers — NOT adaptive     adaptive_l2: false     adaptive_r2:
false     modes: 0   gyroscope:     present: true            # Available on
newer SKUs     axes: 3     resolution_bits: 16     range_dps: 2000
sample_rate_hz: 1000   accelerometer:     present: true     axes: 3
resolution_bits: 16     range_g: 16     sample_rate_hz: 1000   gsr_addon:
supported: false  tier_eligibility:   standard: true   attested: false
# Missing L4 (touchpad), L6 (adaptive triggers)  calibration:
battery_types: ["usb_wired", "battery_full", "battery_low"]
feature_dimensions: 13   feature_mask: 0x0C7F       # Touchpad features
masked out  hid_report_offsets:   lx: 1   ly: 3   rx: 5   ry: 7   l2: 9   r2:
11   gyro_x: 13   gyro_y: 15   gyro_z: 17   accel_x: 19   accel_y: 21
accel_z: 23   touch_x: 0               # N/A — no touchpad   touch_y: 0
# N/A   touch_active: 0          # N/A   buttons: 25  phci:   certified:
false   cert_date: null   cert_version: null
```

NOTE

The Xbox Series X controller is limited to **Standard tier only** due to the absence of a touchpad (L4 = none) and the use of impulse triggers rather than adaptive triggers (L6 = none). Impulse triggers provide vibration feedback but do not support variable resistance measurement required for L6 challenge-response.

9. Security and Privacy Requirements

9.1 Cryptographic Requirements

Function	Algorithm	Usage
Device Signatures	**ECDSA-P256**	All PoAC record signatures; device attestation
Record Hashing	**SHA-256**	`record_hash = SHA-256(raw[0:164])`
Device Identity	**keccak256**	`device_id = keccak256(pubkey)`
Key Generation	On-chip (ATECC608A)	Must occur on secure element; private key never exported
Entropy	Minimum 256-bit	TRNG on secure element for all key generation
Side-Channel Resistance	Constant-time operations	All signing operations must be constant-time to prevent timing attacks

9.2 Privacy Requirements (BP-001 through BP-007)

The VAPI protocol enforces seven Biometric Privacy (BP) requirements. Manufacturers must ensure their firmware and hardware support the following constraints:

ID	Requirement	Description
BP-001	Temporal Biometric Decay	Biometric data weight decays with a **90-day half-life**. All biometric records auto-expire after **180 days**. Firmware must not persistently store biometric data beyond session boundaries.
BP-002	ZK-Attested Consent	All biometric processing requires **zero-knowledge proven consent** using a Poseidon hash commitment. The controller must not transmit biometric-derived data without a valid consent proof.
BP-003	Differential Privacy	Laplacian noise injection with privacy budget **ε ≤ 1.0/year** per player. Firmware feature extraction must support noise injection hooks.
BP-004	K-Anonymity	Cohorts require **K ≥ 5** members before threshold activation. No biometric classification may apply to cohorts smaller than 5.
BP-005	Homomorphic Processing	Separation ratios computed on **encrypted data** using Paillier or CKKS schemes. Raw feature values never

ID	Requirement	Description
		transmitted to comparison endpoints.
BP-006	Shamir Sharing	Biometric identity split across **16 agents** with an **8-of-16 reconstruction threshold**. No single agent possesses sufficient data to reconstruct a player's biometric identity.
BP-007	Ephemeral Sessions	Raw biometric data exists **only in RAM**. Memory must be locked (`mlock`) and securely erased (`memset_secure`) at session end. No swap, no disk persistence.

CRITICAL RULE

No raw biometric data may **ever** leave the controller device or the player's local machine. Only derived hashes, zero-knowledge proofs, and encrypted commitments are transmitted over any network interface. Violation of this rule is grounds for immediate PHCI certification revocation.

9.3 Device Revocation List (DRL)

- Manufacturers must support **Device Revocation List (DRL)** checking in controller firmware.
- The DRL is published on-chain via the `VAPIHardwareCertRegistry` contract.
- Controller firmware must **check the DRL on each session start** (via the VAPI bridge).
- Revoked devices **cannot generate valid PoAC records**. The bridge must refuse to relay PoAC records from revoked devices.

Revocation Reasons:

23. Compromised private key (key material exposure or suspected extraction)
24. Counterfeiting (device_id registered to non-genuine hardware)
25. Failed certification audit (post-market surveillance finding)
26. Manufacturer-initiated recall

10. Production Key Rotation and Revocation Procedures

10.1 Key Rotation Schedule

Key	Slot	Rotation Policy	Notes
Device Identity Key	Slot 0	**NEVER rotated**	Permanent device identity. Configuration zone locked at factory provisioning.
Protocol Signing Key	Slot 1	Annually or on security advisory	Rotated via firmware update mechanism.
Session Keys	Slot 2 (derivation)	Per-session (ephemeral)	Derived at session start, destroyed at session end.
Manufacturer CA Key	Manufacturer HSM	Every 3 years or on compromise	Requires re-provisioning of certificate chain in Slots 3–5.

10.2 Key Rotation Procedure

Step	Action	Responsible Party
1	VAPI Protocol Authority	VAPI Protocol Engineering

Step	Action	Responsible Party
	issues new signing key parameters	
2	Manufacturer pushes firmware update with new key derivation logic	Manufacturer firmware team
3	Controller generates new protocol signing key in Slot 1 on next firmware update	Controller (automatic)
4	Old key remains valid for **90-day transition window**	VAPI verifier network
5	New key registered on-chain via `VAPIHardwareCertRegistry`	VAPI bridge (automatic)
6	After transition window, old key marked deprecated	VAPI Protocol Authority

10.3 Emergency Key Revocation

27. In case of key compromise, the manufacturer issues an **emergency firmware update** disabling the compromised key.
28. The compromised `device_id` is added to the **on-chain DRL immediately**.
29. All PoAC records from the compromised device **after the compromise timestamp** are invalidated retroactively.
30. The affected player must **re-enroll with a new device**. The soulbound VHP (Verified Human Player) credential is transferred to the new `device_id` via a governance-approved migration transaction.
31. **Incident response:** The manufacturer must notify VAPI Protocol Authority **within 24 hours** of discovering or being notified of a key compromise.

10.4 Fail Modes

Tier	Fail Mode	Behavior
Attested	Fail-CLOSED	If secure element communication fails, controller **MUST NOT** generate PoAC records. Player automatically falls to Standard tier. No degraded Attested operation is permitted.
Standard	Fail-OPEN	If a non-critical sensor fails (e.g., touchpad), controller **MAY** continue generating PoAC records with a reduced feature set (partial L4). The VAPI agent fleet handles graceful degradation via the `feature_mask` field.

11. Test Vectors

> **IMPORTANT**
>
> All test vectors in this section use **test-only values**. Never use test keys, test device IDs, or test certificates in production hardware. Production keys must be generated on-chip by the ATECC608A secure element.

11.1 SHA-256 Test Vector (record_hash)

Input: 164-byte PoAC body (all fields zero except those specified below)

```
Field values:   timestamp_ns     = 0x00000000_65A00000   (bytes 0-7)
inference_code   = 0x00                  (byte 8, NOMINAL)   confidence_byte
= 0x00                  (byte 9)   session_id       = 0x00000001
(bytes 10-13)   sequence_number  = 0x00000001            (bytes 14-17)
feature_dim      = 0x000D                (bytes 18-19, value = 13)   reserved
= 0x0000                (bytes 20-21)   anomaly_score    = 0x0000000000000000
(bytes 22-29, IEEE 754 double 0.0)   continuity_score = 0x0000000000000000
(bytes 30-37, IEEE 754 double 0.0)   features[0..11]  = all 0x00
(bytes 38-133, 96 bytes)   device_id        = all 0x00             (bytes
134-149, 16 bytes)   reserved_padding = all 0x00             (bytes 150-163,
14 bytes)  Hex (164 bytes): 00000000 65A00000 00 00 00000001 00000001 000D
```

```
0000 00000000 00000000 00000000 00000000 [96 bytes of 0x00 for features] [16
bytes of 0x00 for device_id] [14 bytes of 0x00 for reserved_padding]
Expected record_hash = SHA-256(body[0:164])
```

Manufacturers must compute SHA-256 over the exact 164-byte body serialization and compare to their implementation output. The hash must use the standard NIST SHA-256 algorithm (FIPS 180-4).

11.2 ECDSA-P256 Test Vector

```
--- TEST KEYS (NEVER USE IN PRODUCTION) ---  Private key (32 bytes, hex):
c9afa9d845ba75166b5c215767b1d6934e50c3db36e89b127b8a622b120f6721  Public key
(65 bytes, uncompressed, hex):   04
60fed4ba255a9d31c961eb74c6356d68c049b8923b61fa6ce669622e60f29fb6
7903fe1008b8bc99a41ae9e95628bc64f2f1b20c2d7e9f5177a3c294d4462299  Message:
164-byte body from Section 11.1 test vector  Signature (r || s, 64 bytes):
(Implementation-dependent due to ECDSA nonce randomness.    Verify by:
ecdsa_verify(pubkey, message, signature) == true)  Verification procedure:
1. Serialize PoAC body to 164 bytes (big-endian)   2. Sign with private key:
sig = ecdsa_p256_sign(privkey, body[0:164])   3. Verify:
ecdsa_p256_verify(pubkey, body[0:164], sig) must return true   4. Construct
228-byte record: body[0:164] || sig[0:64]
```

WARNING

ECDSA signatures are non-deterministic by default (random nonce k). The **exact r, s values will differ** between signing operations. Validate by verifying the signature against the public key, not by comparing raw bytes. If deterministic ECDSA (RFC 6979) is used, the signature will be reproducible.

11.3 Chain Linkage Test Vector

```
Chain of 3 PoAC records:  Record 0 (Genesis):   body[0:164] = test vector
from Section 11.1   record_hash_0 = SHA-256(body_0[0:164])   (No prev_hash —
this is the chain genesis)  Record 1:   body[0:164] = same as Record 0,
except:     sequence_number = 0x00000002     timestamp_ns    =
0x00000000_65A00001   record_hash_1 = SHA-256(body_1[0:164])   Verification:
prev_hash field must equal record_hash_0  Record 2:   body[0:164] = same as
Record 0, except:     sequence_number = 0x00000003     timestamp_ns    =
0x00000000_65A00002   record_hash_2 = SHA-256(body_2[0:164])   Verification:
prev_hash field must equal record_hash_1  Chain validation:   For each
record[i] where i > 0:     assert SHA-256(body[i-1][0:164]) ==
record[i].prev_hash
```

11.4 device_id Computation Test Vector

```
Input: P256 public key (65 bytes, uncompressed with 0x04 prefix):   04
60fed4ba255a9d31c961eb74c6356d68c049b8923b61fa6ce669622e60f29fb6
7903fe1008b8bc99a41ae9e95628bc64f2f1b20c2d7e9f5177a3c294d4462299  Step 1:
Compute keccak256 over the full 65-byte uncompressed public key   full_hash =
keccak256(pubkey_65_bytes)   // Result: 32 bytes  Step 2: Truncate to 128
bits (16 bytes)   device_id = full_hash[0:16]  Verification:   1. Hash the
65-byte public key with keccak256   2. Take the first 16 bytes of the result
3. This 16-byte value is the device_id stored at offset 134 in the PoAC body
4. The full 32-byte keccak256 hash is used for on-chain DID:
did:io:0x{hex(full_hash)}
```

12. Compliance Summary Table

Requirement	Attested Tier	Standard Tier	Notes
ATECC608A Secure Element	**REQUIRED**	Recommended	On-chip key generation, never exported
6-Axis IMU (Gyro + Accel)	**REQUIRED**	**REQUIRED**	16-bit, 1000 Hz, ±2000 dps / ±16 g
Touchpad (Capacitive)	**REQUIRED**	Optional	1920×1080 minimum, multi-touch
Adaptive Triggers (L6)	**REQUIRED**	Not required	Variable resistance measurement, 3 modes min
GSR Grip Module	Optional	Optional	ESP32-S3 BLE addon
VAPI Firmware (256 KB)	**REQUIRED**	**REQUIRED**	PoAC builder, crypto, chain logic
USB 1000 Hz Polling	**REQUIRED**	Optional (250 Hz BLE OK)	64-byte HID report
PITL L0–L5	**REQUIRED** (full)	**REQUIRED** (minimum)	Stick, button, gyro, accel, touchpad, composite
PITL L6 (Adaptive Trigger)	**REQUIRED** (full)	Not required	Challenge-response resistance measurement

Requirement	Attested Tier	Standard Tier	Notes
p99 Latency	< 2 ms	< 5 ms	End-to-end HID report timing
Jitter (std dev)	< 0.5 ms	< 1 ms	Inter-report interval standard deviation
Packet Loss	0%	< 0.1%	Over 10,000 consecutive reports
Clock Drift	< 1 ppm	—	Attested tier only
PoAC Wire Format (228 bytes)	**REQUIRED**	**REQUIRED**	FROZEN — no modifications permitted
ECDSA-P256 Signatures	**REQUIRED**	**REQUIRED**	All PoAC records must be signed
SHA-256 Chain Hashing	**REQUIRED**	**REQUIRED**	`record_hash = SHA-256(body[0:164])`
keccak256 Device ID	**REQUIRED**	**REQUIRED**	`device_id = keccak256(pubkey)`
DRL Checking	**REQUIRED**	**REQUIRED**	Check on every session start
BP-001 through BP-007	**REQUIRED**	**REQUIRED**	All biometric privacy requirements apply to both tiers
Fail Mode	Fail-CLOSED	Fail-OPEN	Attested: no PoAC on SE failure. Standard: degrade gracefully.
Calibration Sessions	N ≥ 50	N ≥ 30	Per controller model for threshold derivation
PHCI Certification	**REQUIRED**	Self-registered	Submit 3 units + documentation
On-Chain Attestation	**REQUIRED**	Optional	Via `VAPIHardwareCertRegistry`
Controller Profile YAML	**REQUIRED**	**REQUIRED**	Per Section 8 schema

Requirement	Attested Tier	Standard Tier	Notes
Incident Response (24 hr)	**REQUIRED**	**REQUIRED**	Notify VAPI within 24 hours of key compromise

Appendix A: Glossary

Term	Definition
PoAC	**Proof of Authentic Controller.** A 228-byte cryptographically signed record attesting that controller input originated from a specific physical device held by a specific human player. Forms a tamper-evident chain via SHA-256 hash linkage.
PITL	**Physical Input Trust Layers.** A layered classification framework (L0–L6) where each layer analyzes a different physical signal from the controller: L0 (stick dynamics), L1 (button timing), L2 (gyroscope), L3 (accelerometer), L4 (touchpad), L5 (composite multi-layer), L6 (adaptive trigger challenge-response).
VHP	**Verified Human Player.** A soulbound on-chain credential proving that a player has been biometrically verified as a unique human through sufficient VAPI calibration sessions. Non-transferable (soulbound).
PHCI	**Physical Hardware Certification Initiative.** The formal certification program for game controllers seeking VAPI compatibility. Defines hardware requirements, testing procedures, and tier eligibility criteria.
DRL	**Device Revocation List.** An on-chain registry of device IDs that have been revoked due to key compromise, counterfeiting, or failed certification audits. Maintained via the `VAPIHardwareCertRegistry` contract.

Term	Definition
ioID	**IoTeX Decentralized Identity.** The on-chain identity framework used by VAPI for device registration. Each controller receives a DID in the format `did:io:0x{device_id}`.
ioSwarm	**IoTeX Swarm Network.** The decentralized compute and data infrastructure on IoTeX that hosts VAPI's agent fleet, calibration models, and privacy-preserving biometric processing.
VAPI Bridge	The desktop application running on the player's PC that communicates with the controller via USB/BLE HID, performs local inference, builds PoAC records, and relays attestations to the IoTeX blockchain.
Attested Tier	The highest VAPI certification tier requiring full L0–L6 PITL coverage, PHCI certification, adaptive triggers, 1000 Hz transport, and N≥50 calibration sessions. Provides maximum anti-cheat assurance.
Standard Tier	The baseline VAPI certification tier requiring L0–L5 PITL coverage (minimum), 250 Hz transport acceptable, and N≥30 calibration sessions. Self-registered without mandatory PHCI testing.
Secure Element	A tamper-resistant hardware component (e.g., ATECC608A) that performs cryptographic operations and stores private keys in a way that prevents extraction, even with physical access to the device.
Chain Link	The SHA-256 hash of a PoAC record body (bytes[0:164]) that connects consecutive records into a tamper-evident chain. Expressed as `record_hash = SHA-256(raw[0:164])`.
GSR	**Galvanic Skin Response.** A measure of skin electrical conductance, used as an optional biometric signal. Captured via electrode-equipped grip modules.

Term	Definition
Poseidon Hash	A ZK-SNARK-friendly hash function used in VAPI for zero-knowledge consent proofs (BP-002). Efficient to verify inside arithmetic circuits.

Appendix B: Reference Contracts

The following IoTeX L1 smart contracts are the primary on-chain interfaces that manufacturers and their firmware interact with during device provisioning and operation.

Contract	Purpose	Key Functions
VAPIioIDRegistry	Device registration and identity management	`registerDevice(deviceId, pubkey, certHash)` — Registers a new controller on-chain. `getDevice(deviceId)` — Returns device metadata (pubkey, manufacturer, certLevel). DID format: `did:io:0x{device_id}`
VAPIHardwareCertRegistry	Hardware certification and revocation	`certifyDevice(deviceId, certLevel)` — Sets certification level (1=basic, 2=standard, 3=attested). `revokeDevice(deviceId, reason)` — Adds device to DRL. `isRevoked(deviceId)` — Checks DRL status. `getCertLevel(deviceId)` — Returns current certification level.
VAPIProtocolLens	Composable eligibility gate	`isFullyEligible(deviceId)` — Returns `true` if device is registered, certified, not revoked, and has valid calibration. Composable with other on-chain protocols for gating access to

Contract	Purpose	Key Functions
		tournaments, ranked play, or data marketplace participation.
PoACVerifier	On-chain ECDSA-P256 signature verification	`verifyPoAC(record, pubkey)` — Verifies the ECDSA-P256 signature of a 228-byte PoAC record against the device's registered public key. Uses EVM precompile at address `0x0100` for gas-efficient P256 verification.

NOTE

All contract interactions during device provisioning (Section 4.2, Steps 4–5) are performed by the VAPI bridge application on behalf of the manufacturer. Manufacturers do not need to deploy or manage smart contracts directly. Contract ABIs and integration documentation are provided separately under NDA.
